

# Revisão de Jetpack Compose

Aplicação de exemplo: `ComposeReview`

Pedro Oliveira

2 de julho de 2026

## Sumário

|          |                                                            |          |
|----------|------------------------------------------------------------|----------|
| <b>1</b> | <b>Introdução</b>                                          | <b>2</b> |
| <b>2</b> | <b>Estrutura do projeto</b>                                | <b>2</b> |
| <b>3</b> | <b>Os seis conceitos, um a um</b>                          | <b>3</b> |
| 3.1      | <code>@Composable</code> . . . . .                         | 3        |
| 3.2      | Composição e recomposição . . . . .                        | 3        |
| 3.3      | <code>state</code> . . . . .                               | 4        |
| 3.4      | <code>remember</code> . . . . .                            | 4        |
| 3.5      | <code>mutableStateOf</code> . . . . .                      | 5        |
| 3.6      | UI declarativa . . . . .                                   | 5        |
| <b>4</b> | <b>Fluxo de dados: do toque na tela até a recomposição</b> | <b>6</b> |
| <b>5</b> | <b>Conclusão</b>                                           | <b>6</b> |

## 1 Introdução

Este documento acompanha o projeto Android `jetpack-compose-review/`, uma aplicação mínima construída com Jetpack Compose para revisar, na prática, seis conceitos fundamentais do framework:

- `@Composable`
- Composição e recomposição
- `state`
- `remember`
- `mutableStateOf`
- UI declarativa

A aplicação tem duas telas simples em uma única tela rolável: um **contador** e uma **lista de tarefas**. Cada uma foi escolhida por exercitar um subconjunto diferente dos conceitos acima, mantendo o código pequeno o suficiente para ler de uma vez só.

## 2 Estrutura do projeto

O projeto segue a estrutura padrão de um módulo Android/Gradle:

```
jetpack-compose-review/
|-- build.gradle.kts           # configuracao raiz (plugins)
|-- settings.gradle.kts       # declara o modulo :app
|-- app/
|   |-- build.gradle.kts      # dependencias do Compose/Material3
|   '-- src/main/
|       |-- AndroidManifest.xml
|       |-- java/com/pedro/composereview/
|           |-- MainActivity.kt    # Activity + todos os Composables
|           '-- ui/theme/
|               |-- Color.kt
|               |-- Theme.kt
|               '-- Type.kt
|       '-- res/values/
|           |-- strings.xml
|           '-- themes.xml
'-- docs/
    '-- revisao-jetpack-compose.tex    # este documento
```

Diferente de um projeto Android tradicional baseado em Views e XML de layout, aqui *toda* a interface é descrita em Kotlin dentro de `MainActivity.kt`, usando funções anotadas com `@Composable`. Os arquivos em `ui/theme/` apenas definem cores, tipografia e o `MaterialTheme` usado pela aplicação — não contém lógica de tela.

## 3 Os seis conceitos, um a um

### 3.1 @Composable

`@Composable` é a anotação que transforma uma função Kotlin comum em uma **unidade de UI** que o compilador do Compose sabe como rastrear, (re)executar e otimizar. Uma função marcada com `@Composable` pode chamar outras funções `@Composable`, ler estado (`State<T>`) e emitir elementos de UI — mas não pode ser chamada de código Kotlin comum fora desse contexto.

No projeto, todas as telas são funções `@Composable`: `ReviewApp`, `SectionTitle`, `CounterCard`, `TodoCard`, `TaskList` e `TaskRow`. Exemplo:

```
1 @Composable
2 private fun SectionTitle(text: String) {
3     Text(
4         text = text,
5         fontWeight = FontWeight.Bold,
6         style = MaterialTheme.typography.titleMedium
7     )
8 }
```

Listing 1: Uma função `@Composable` simples e reutilizável

Note que `SectionTitle` não devolve uma `View`: ela apenas *descreve* que, naquele ponto da árvore, deve existir um `Text` com determinado estilo. Quem decide *quando* e *como* desenhar isso é o runtime do Compose — esse é o cerne da UI declarativa (Seção 3.6).

### 3.2 Composição e recomposição

**Composição** é o processo de executar as funções `@Composable` e montar a árvore de UI resultante (quais elementos existem, com quais parâmetros, aninhados de que forma). Isso acontece na primeira renderização da tela.

**Recomposição** é o processo de executar *de novo* apenas as funções `@Composable` cujas leituras de estado mudaram, para atualizar a árvore — sem recriar a `Activity` nem reconstruir a UI do zero. O Compose rastreia, para cada composável, quais objetos de `State` ele leu durante a composição; quando um desses valores muda, apenas os composáveis que o leram (e, quando necessário, seus descendentes) são recompostos.

Para tornar esse processo visível, `CounterCard` mantém um contador de recomposições (**recompositions**) que *não* é um `State` — é uma variável comum guardada com `remember`. Ela é incrementada toda vez que a função roda de novo, mas sua leitura não "assina" a composição atual (diferente de um `mutableStateOf`), então ela não causa recomposições adicionais por conta própria — apenas reflete quantas vezes o `count` mudou até agora:

```
1 @Composable
2 fun CounterCard() {
3     var count by remember { mutableStateOf(0) }
4     val recompositions = remember { intArrayOf(0) }
5     recompositions[0] += 1
6
7     Card(/* ... */) {
8         Column(/* ... */) {
9             Text("Valor: $count")
10            Text("Esta secao recompos ${recompositions[0]} vez(es)")
11            Row {
12                Button(onClick = { count++ }) { Text("+1") }
13                Button(onClick = { count-- }) { Text("-1") }
14                TextButton(onClick = { count = 0 }) { Text("Resetar") }
15            }
16        }
17    }
```

```
17 }  
18 }
```

Listing 2: Tornando a recomposição visível na tela

A cada toque em "+1", "-1" ou "Resetar": (1) `count` muda; (2) o Compose marca `CounterCard` como inválido, pois foi ele quem leu `count` durante a composição; (3) `CounterCard` é executado de novo (recomposição); (4) `recompositions[0]` é incrementado e o novo valor aparece na tela — evidenciando que a recomposição de fato ocorreu.

### 3.3 state

Em Compose, **state** (estado) é qualquer valor que, ao mudar, deveria fazer a UI se atualizar. Ele é representado pela interface `State<T>` (e sua variante mutável, `MutableState<T>`). O Compose observa leituras de `State` feitas dentro de um escopo de composição e usa isso para saber o que recompor quando o valor muda.

O projeto usa estado em dois lugares com naturezas diferentes:

- **Estado local e primitivo:** `count` (um `Int`) em `CounterCard` e `input` (uma `String`) em `TodoCard`.
- **Estado de coleção:** `tasks`, uma `SnapshotStateList<String>` criada com `mutableStateListOf(...)`, que notifica a composição quando itens são adicionados ou removidos (`tasks.add(...)`, `tasks.remove(...)`), sem precisar reatribuir a variável inteira.

Também é demonstrado o padrão de **state hoisting**: em vez de cada composável filho guardar seu próprio estado, `TodoCard` concentra o estado (`input` e `tasks`) e o repassa como parâmetros e callbacks para `TaskList` e `TaskRow`, que ficam *stateless* — apenas recebem dados e informam eventos para cima:

```
1 @Composable  
2 fun TaskList(tasks: List<String>, onRemove: (String) -> Unit) {  
3     Column {  
4         tasks.forEach { task ->  
5             TaskRow(task = task, onRemove = { onRemove(task) })  
6         }  
7     }  
8 }  
9  
10 @Composable  
11 fun TaskRow(task: String, onRemove: () -> Unit) {  
12     Row {  
13         Text(text = task, modifier = Modifier.weight(1f))  
14         TextButton(onClick = onRemove) { Text("Remover") }  
15     }  
16 }
```

Listing 3: State hoisting: pai com estado, filhos sem estado

Essa separação torna `TaskList` e `TaskRow` fáceis de reutilizar, testar e pré-visualizar isoladamente, porque não dependem de *onde* o estado mora — apenas do que recebem por parâmetro.

### 3.4 remember

Funções `@Composable` podem ser executadas várias vezes (recompostas). Sem alguma forma de persistir valores entre essas execuções, qualquer variável local seria recriada do zero a cada recomposição — um contador sempre voltaria a zero, por exemplo.

**remember** resolve isso: ele guarda um valor na memória interna da composição (a *slot table*) e o devolve nas recomposições seguintes, em vez de recalculá-lo. O valor só é recriado se o composável sair completamente da árvore (por exemplo, some da tela) e entrar de novo, ou se as chaves passadas a variações como **remember(key)** mudarem.

No projeto, **remember** aparece em toda inicialização de estado:

```
1 var count by remember { mutableStateOf(0) }
2 val recompositions = remember { intArrayOf(0) }
3 var input by remember { mutableStateOf("") }
4 val tasks = remember { mutableStateListOf("Revisar @Composable", "Estudar remember") }
```

Listing 4: remember preservando estado entre recomposições

Em todos os casos, a lambda passada para **remember** só é executada **uma vez** (na primeira composição); nas recomposições seguintes, **remember** simplesmente devolve o valor já existente.

### 3.5 mutableStateOf

`mutableStateOf(valorInicial)` cria um objeto `MutableState<T>` observável: sempre que sua propriedade `.value` é reatribuída, o Compose sabe que precisa invalidar (e depois recompor) qualquer composável que tenha lido esse valor durante a composição.

Combinado com o delegate de propriedade `by` (do pacote `androidx.compose.runtime`, via `getValue/setValue`), é possível tratar o estado quase como uma variável comum:

```
1 var count by remember { mutableStateOf(0) }
2 // equivalente, sem o delegate 'by':
3 // val countState = remember { mutableStateOf(0) }
4 // countState.value++
5
6 count++ // lê e escreve em countState.value internamente
```

Listing 5: mutableStateOf com o delegate 'by'

O mesmo padrão é usado para `input` em `TodoCard` (`var input by remember { mutableStateOf("") }`), atualizado a cada tecla digitada via `onValueChange = { input = it }` do `OutlinedTextField`. Para a lista de tarefas, o equivalente para coleções é usado: `mutableStateListOf(...)`, que devolve uma lista cujas operações de mutação (`add`, `remove`) já disparam recomposição sozinhas.

### 3.6 UI declarativa

Em UI **imperativa** (o modelo clássico de Views no Android), o desenvolvedor descreve *passo a passo como transformar* a UI de um estado para outro: cria uma `View`, chama `findViewById`, depois `setText(...)`, `setVisibility(...)`, etc., sempre que algo muda.

Em UI **declarativa** (o modelo do Compose), o desenvolvedor descreve apenas *como a UI deve ser para um dado estado atual* — uma função pura, em essência: `UI = f(state)`. Quando o estado muda, a função é executada de novo (recomposição) e o Compose calcula sozinho a diferença necessária na árvore de UI, sem que o código precise dizer "esconda este botão" ou "atualize aquele texto" manualmente.

Isso fica evidente em `CounterCard`: o texto `"Valor: $count"` não é atualizado com um comando imperativo do tipo `textView.setText(...)`. Em vez disso, a função inteira é reexecutada com o novo valor de `count`, e a nova string `"Valor: $count"` é simplesmente o resultado dessa execução:

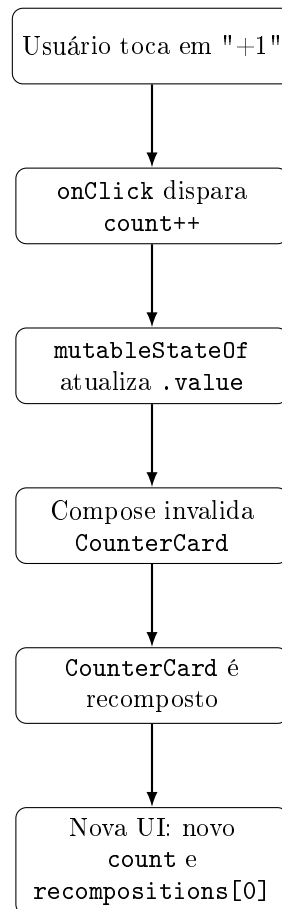
```
1 Text("Valor: $count", style = MaterialTheme.typography.headlineSmall)
```

Listing 6: UI = f(state): o texto e apenas funcao do estado atual

Não existe, em nenhum lugar do código, uma chamada explícita para "atualizar o texto" — o Compose garante isso automaticamente porque `Text` leu `count` durante a composição.

## 4 Fluxo de dados: do toque na tela até a recomposição

O diagrama abaixo resume o ciclo que ocorre a cada toque no botão "+1" do contador:



O mesmo ciclo se repete para a lista de tarefas, com a diferença de que o estado (`tasks`) mora em `TodoCard` e é apenas *lido* por `TaskList/TaskRow` — por isso, ao adicionar ou remover uma tarefa, é `TodoCard` (e os composables que efetivamente leem `tasks`) que recompõe, não a árvore inteira do aplicativo.

## 5 Conclusão

A aplicação `ComposeReview` concentra, em menos de 200 linhas de Kotlin, os seis conceitos revisados: funções `@Composable` formam a árvore de UI; `state` (via `mutableStateOf` e `mutableStateListOf`) guarda os dados que podem mudar; `remember` preserva esse estado entre execuções; e a composição/recomposição é o mecanismo que mantém a tela sincronizada com o estado, seguindo o modelo declarativo `UI = f(state)` — sem nenhuma chamada imperativa de atualização de tela.